

Application Observability

In Meican

Table of Content

- What is Application Observability ?
- Tools and best practices
 - Logs
 - Metrics
 - Traces
 - Alerts

Application Observability

- Understand what's happening
- Logs, metrics, and traces
- Application Performance Monitoring (APM)
 - OpenCensus
 - OpenTelemetry

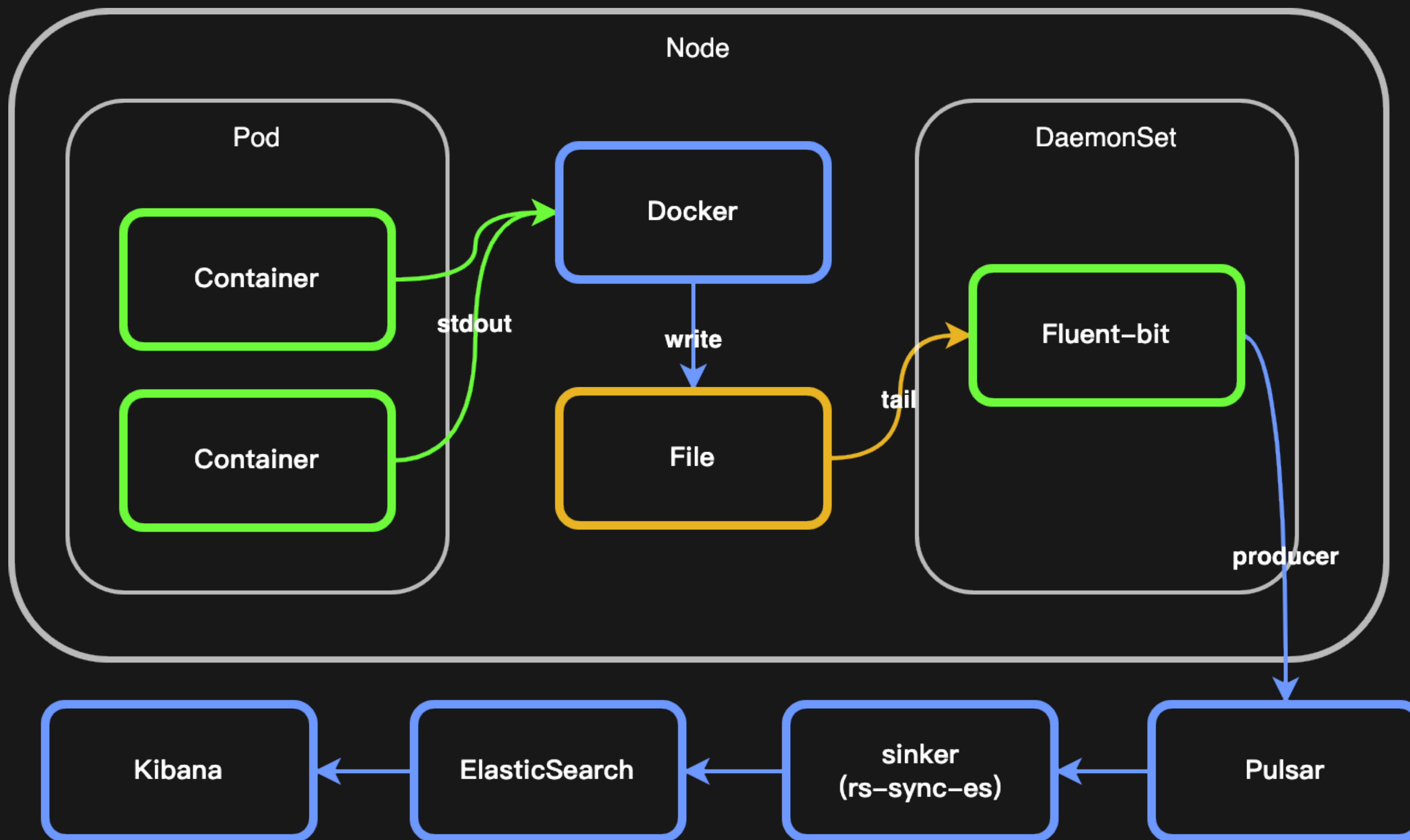
Benefits

- Performance monitoring:
 - "If you can't measure it you can't improve it."
- DevOps and SRE:
 - Supporting infrastructure and resources
- Catch and resolve issues:
 - Closed loop alerting
- etc...

Logs

By Elasticsearch & Fluentbit & Kibana (EFK)

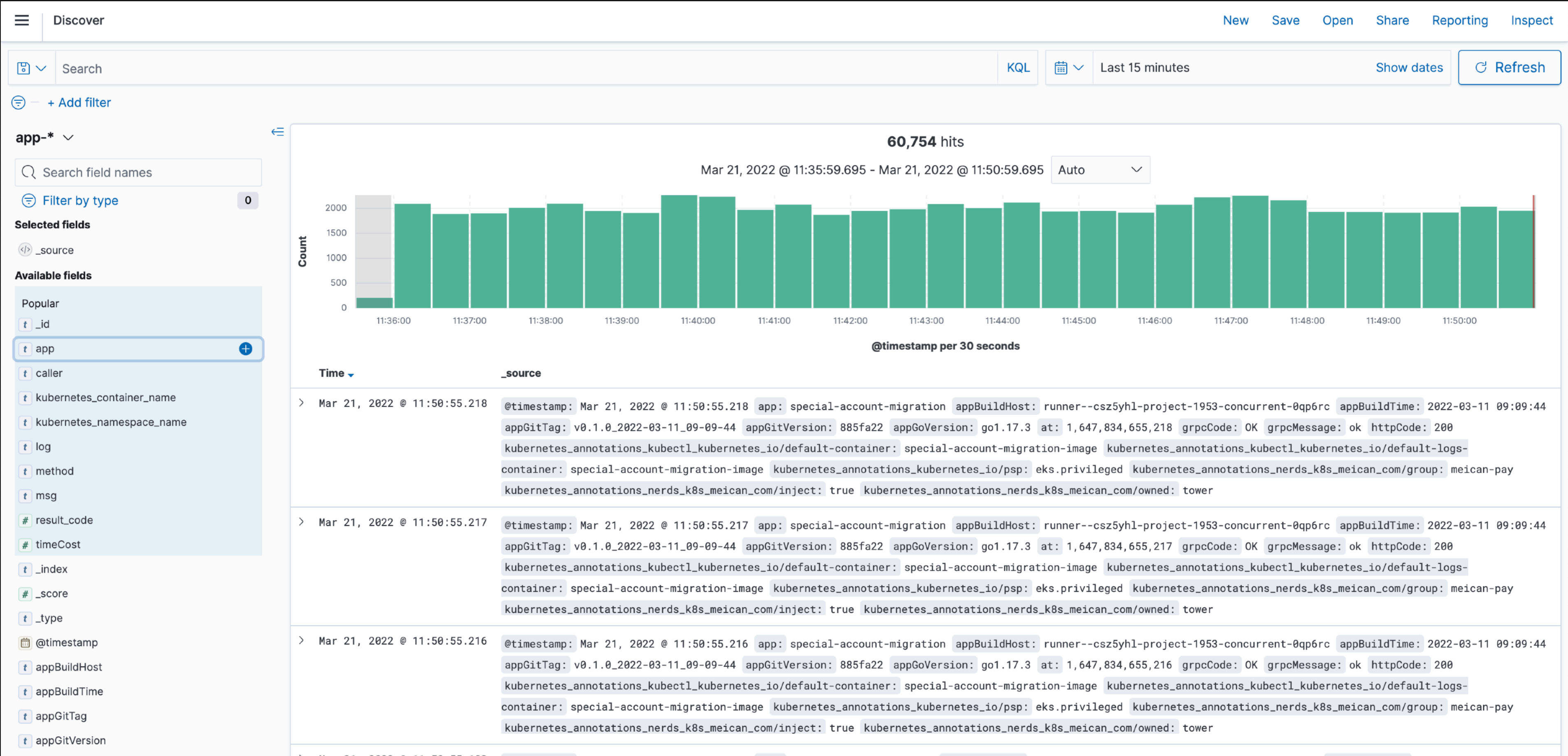
Log Collection



go.planetmeican.com/nerds/log



- Structured
- Leveled
- Proper metadata tags



Kibana

<https://log-nw.planetmeican.com>

Query Logs

app: pagerduty × + Add filter

EDIT FILTER

Edit as Query DSL

Field

app

Operator

is

Value

pagerduty

☐ Create custom label?

Cancel Save

app: pagerduty × error: exists × + Add filter

app-*,eksfan-logsta

Search field names

Filter by type

Selected fields

</> _source

Available fields

Popular

t app

EDIT FILTER

Edit as Query DSL

Field

error

Operator

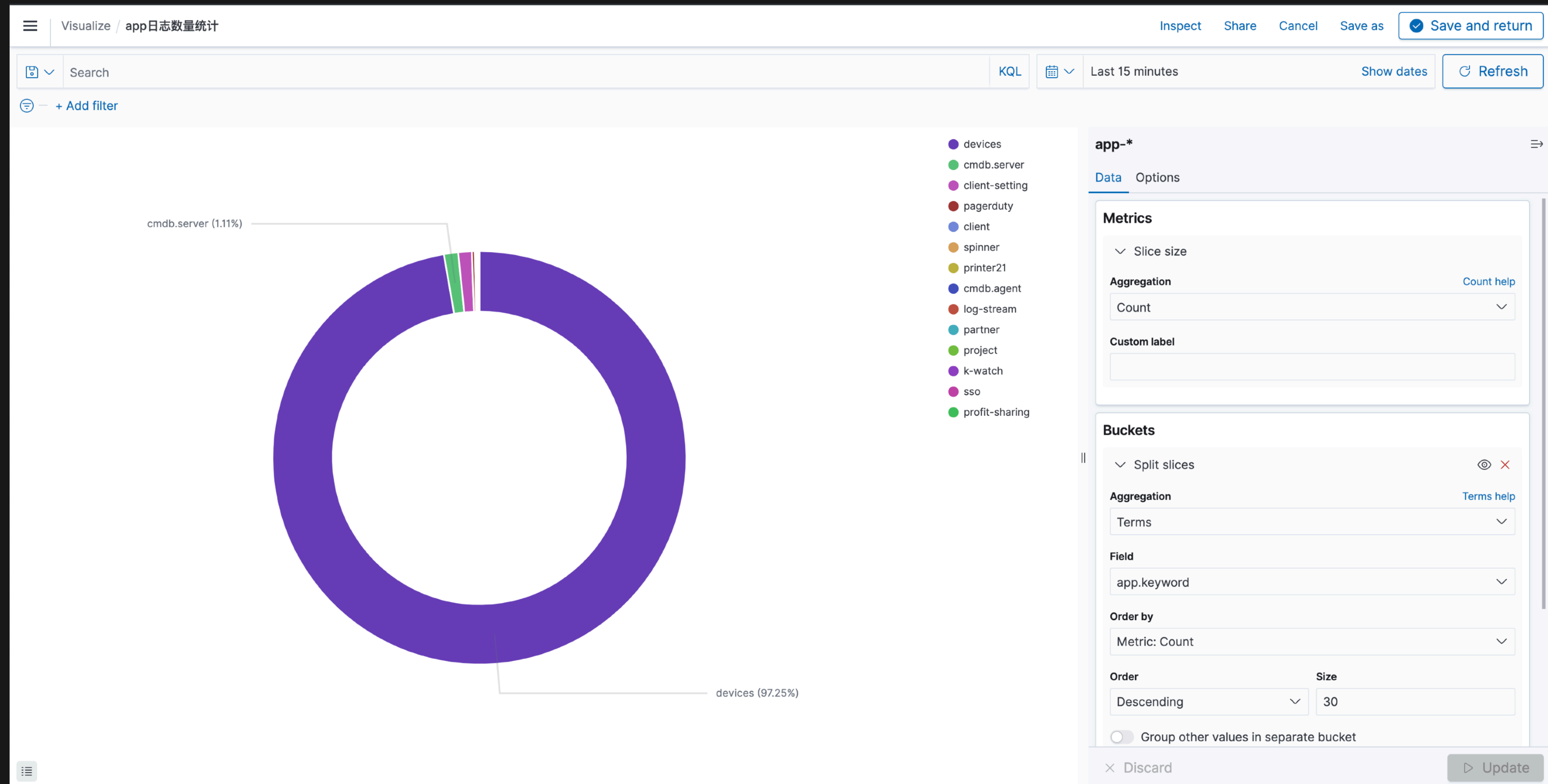
exists

app-*,eksfan-logstash-*

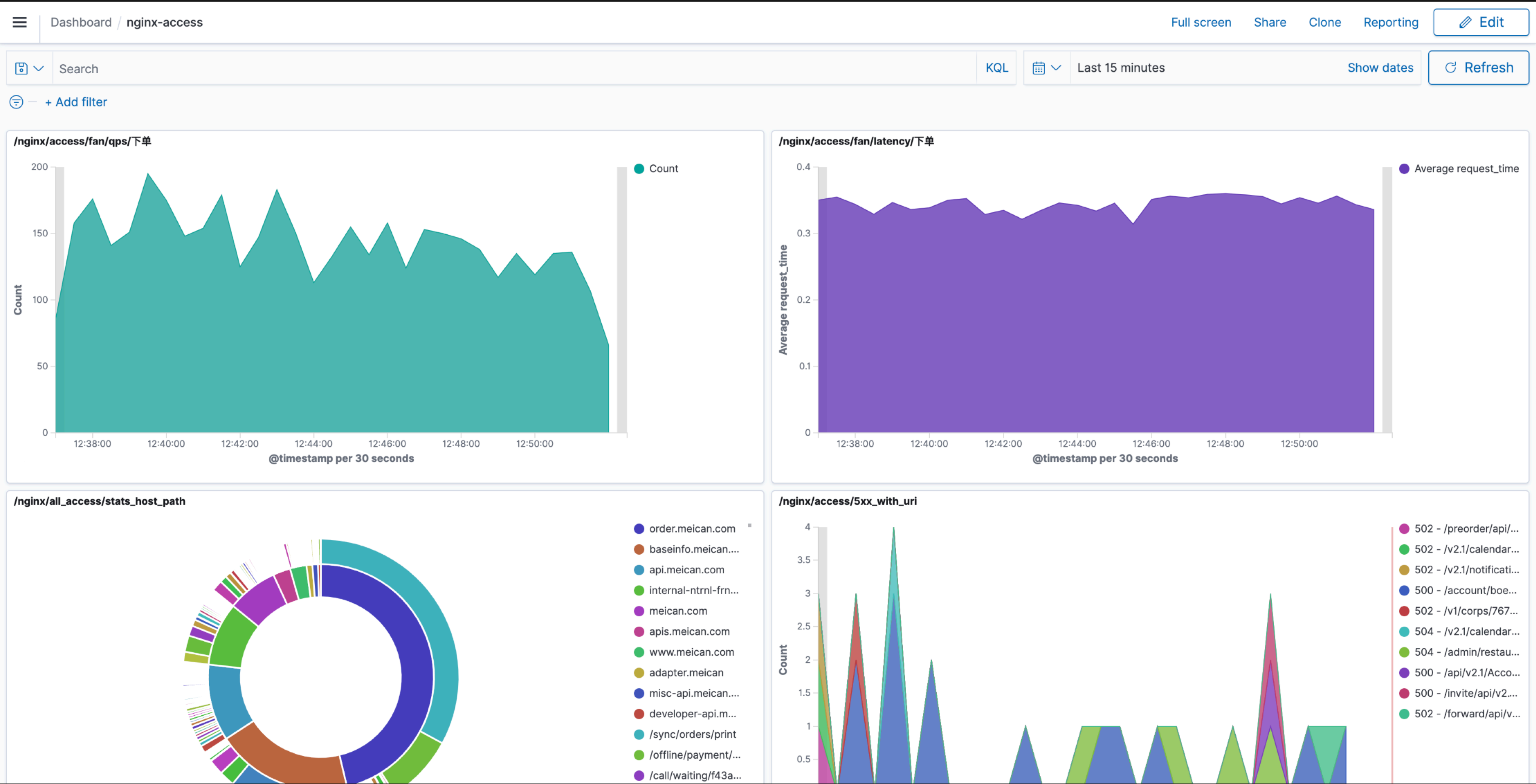
☐ Create custom label?

Cancel Save

Draw Visualization

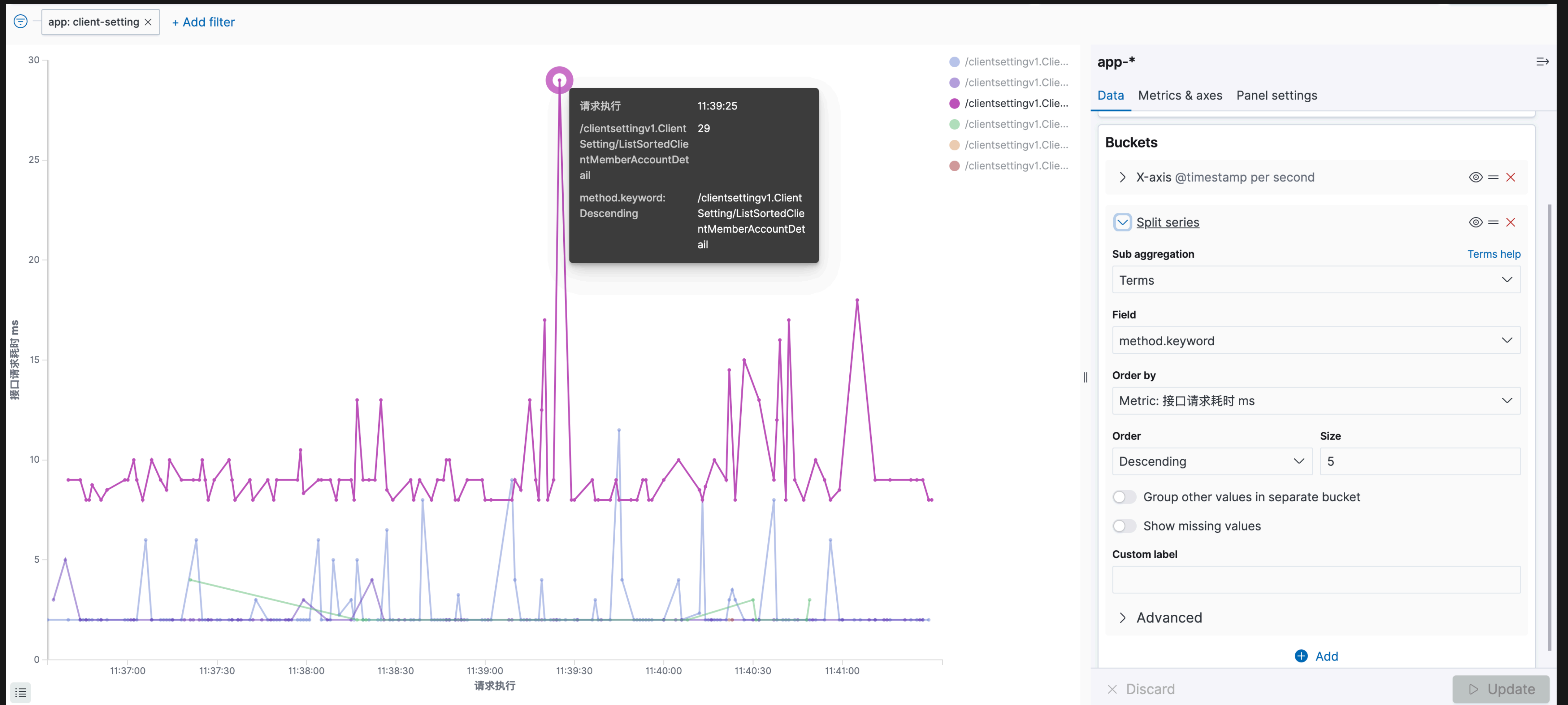


Combine Dashboard



Example: Nginx Access

QPS Example



Best Practices

- Print stack: error verbose
- Clear key-value pairs
- About log.Any:
 - Avoid using it (unpredictable)
 - Different type, different key (ES constraint)
- Ban large value (ES constraint)
- Turn off debug in production:
 - <https://grafana-nw1.planetmeican.com/d/szlaMeBz-logging-pulsar/pulsar-elasticsearch-sync-rc?viewPanel=105&orgId=1&from=now-15m&to=now>

Verbose Error

```
dashboard, err := p.infraRepo.ReconcileGrafanaDashboard(ctx, grafanaServer, board.UID, board.Slug)
if err != nil {
    return errors.Wrapf(err, format: "infraRepo.ReconcileGrafanaDashboard for %s/%s failed", namespace, project)
}
if err := p.appRepo.AttachClusterResource(ctx, subset.ID, dashboard.ID); err != nil {
    return errors.Wrapf(err, format: "appRepo.AttachClusterResource for %s/%s failed", namespace, project)
}
```

go.planetmeican.com/nerds/errors

```
if err != nil {
    log.Error(msg: "internalReconcileGrafanaDashboardForApplication failed",
    log.String(key: "grafanaNamespace", namespace),
    log.String(key: "grafanaProject", project),
    log.ErrField(err),
    )
}
```

go.planetmeican.com/nerds/log

Verbose Error

```

WithinTx.fn: grafanaClient.ReconcileDashboard: client.Search for sdkFolder Apps: HTTP error 503: returns {"message":"failure to get a peer from the ring-balancer"}
(1) attached stack trace
-- stack trace:
| cmdb/internal/utils/dbutils.WithinTx
|   /go/src/internal/utils/dbutils/tx.go:25
| cmdb/internal/service/grpcprovider.(*cmdbServerProvider).AsyncReconcileGrafanaDashboardForApplication.func1
|   /go/src/internal/service/grpcprovider/application.go:610
| cmdb/internal/utils/recovery.internalRecover
|   /go/src/internal/utils/recovery/recovery.go:44
| cmdb/internal/utils/recovery.WithRecovery
|   /go/src/internal/utils/recovery/recovery.go:49
Wraps: (2) WithinTx.fn
Wraps: (3) attached stack trace
-- stack trace:
| cmdb/internal/service/grpcprovider.(*cmdbServerProvider).internalReconcileGrafanaDashboardForApplication
|   /go/src/internal/service/grpcprovider/application.go:633
| [...repeated from below...]
Wraps: (4) grafanaClient.ReconcileDashboard
Wraps: (5) attached stack trace
-- stack trace:
| cmdb/internal/infra/grafana.(*client).ReconcileFolder
|   /go/src/internal/infra/grafana/grafana.go:117
| cmdb/internal/infra/grafana.(*client).ReconcileDashboard
|   /go/src/internal/infra/grafana/grafana.go:159
| cmdb/internal/service/grpcprovider.(*cmdbServerProvider).internalReconcileGrafanaDashboardForApplication
|   /go/src/internal/service/grpcprovider/application.go:631
| cmdb/internal/service/grpcprovider.(*cmdbServerProvider).AsyncReconcileGrafanaDashboardForApplication.func1.1
|   /go/src/internal/service/grpcprovider/application.go:611
| cmdb/internal/utils/dbutils.WithinTx
|   /go/src/internal/utils/dbutils/tx.go:24
| cmdb/internal/service/grpcprovider.(*cmdbServerProvider).AsyncReconcileGrafanaDashboardForApplication.func1
|   /go/src/internal/service/grpcprovider/application.go:610
| cmdb/internal/utils/recovery.internalRecover
|   /go/src/internal/utils/recovery/recovery.go:44
| cmdb/internal/utils/recovery.WithRecovery
|   /go/src/internal/utils/recovery/recovery.go:49
| runtime.goexit
|   /usr/local/go/src/runtime/asm_amd64.s:1581
Wraps: (6) client.Search for sdkFolder Apps
Wraps: (7) HTTP error 503: returns {"message":"failure to get a peer from the ring-balancer"}
Error types: (1) *errors.withStackError (2) *errors.withPrefixError (3) *errors.withStackError (4) *errors.withPrefixError (5) *errors.withStackError (6) *errors.withPrefixError (7) *errors.errorString

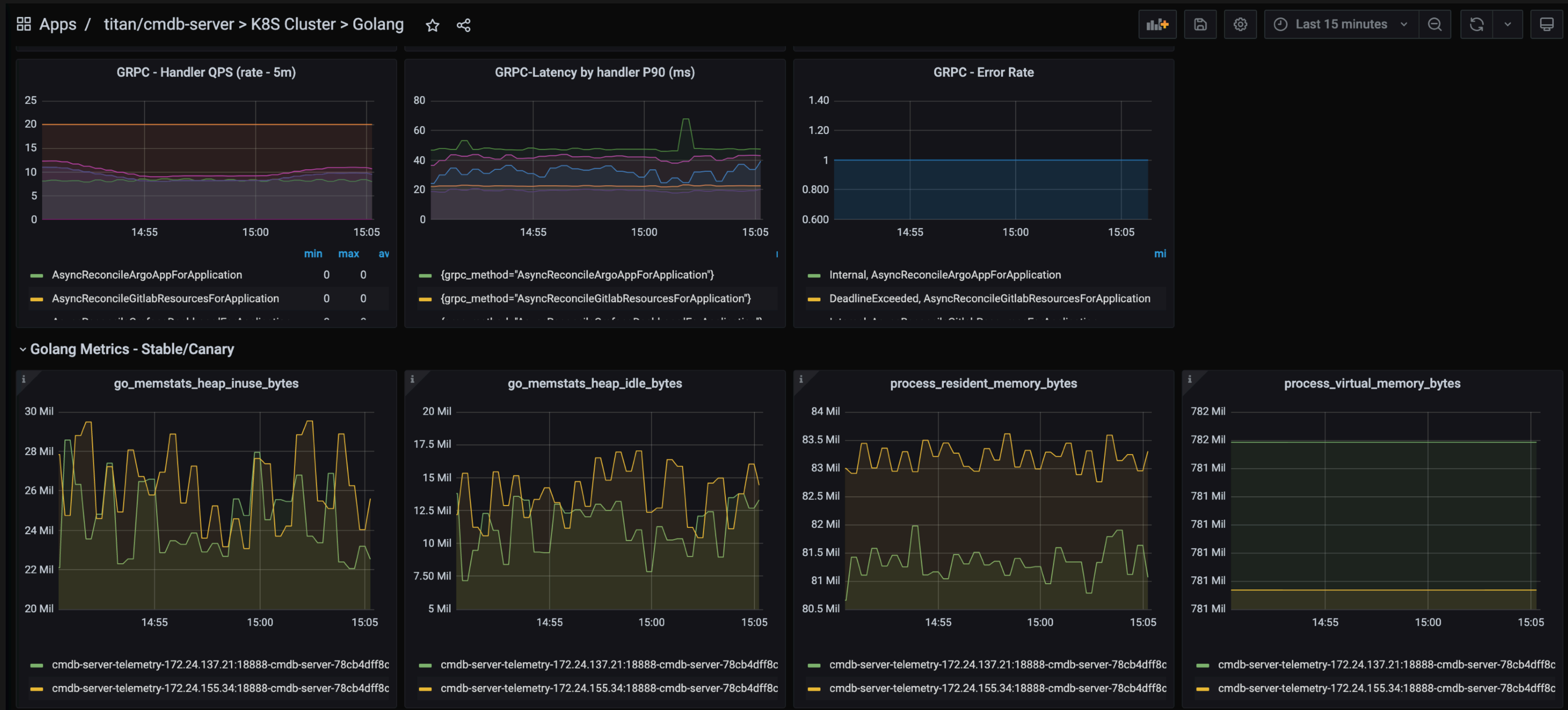
```

Ongoing projects

- Machine learning for log analysis:
 - OOPS-627
 - OOPS-628
 - OOPS-629
- Trigger alerts by logs:
 - <https://gitlab.planetmeican.com/titan/log-stream>
 - <https://planetmeican.atlassian.net/wiki/spaces/DEV/pages/386564113/LogStream>

Metrics

By Prometheus & Thanos



Grafana

<https://grafana-nw.planetmeican.com>

Prometheus Metrics

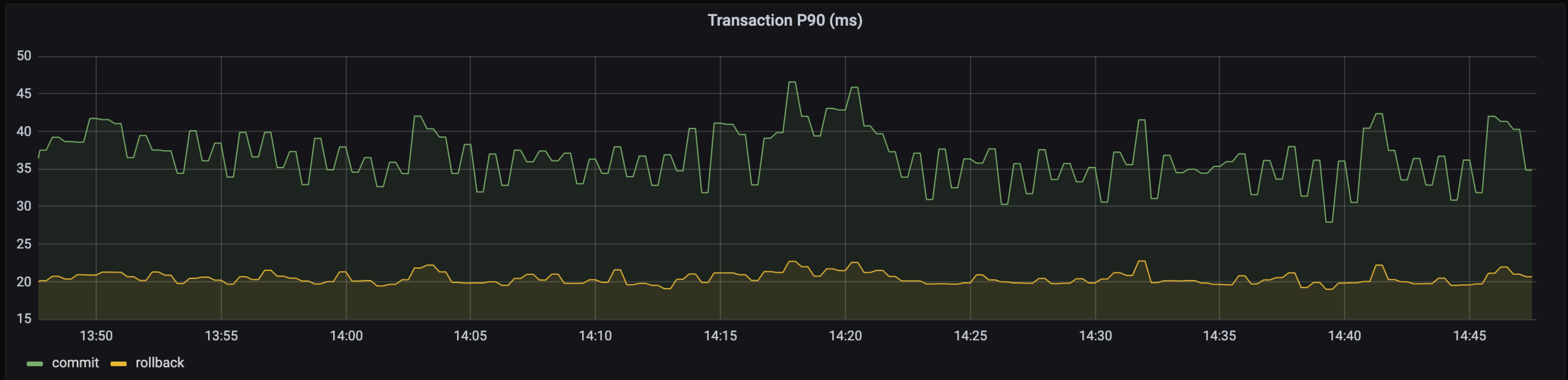
```
# HELP gdb_dbm_transaction_seconds Histogram of dbmanager transaction hold seconds, including commit and rollback
# TYPE gdb_dbm_transaction_seconds histogram
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="0.005"} 1.005145e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="0.01"} 2.907767e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="0.025"} 4.678704e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="0.05"} 5.108097e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="0.1"} 5.134252e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="0.25"} 5.18212e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="0.5"} 5.191299e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="1"} 5.193488e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="2.5"} 5.194239e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="5"} 5.194353e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="10"} 5.19441e+06
gdb_dbm_transaction_seconds_bucket{dataNode="rw",tx_status="commit",le="+Inf"} 5.194716e+06
gdb_dbm_transaction_seconds_sum{dataNode="rw",tx_status="commit"} 216041.17646658595
gdb_dbm_transaction_seconds_count{dataNode="rw",tx_status="commit"} 5.194716e+06
```

<metric name>{<label name>=<label value>, ...}

Prometheus Metric Types

- **Counter**
- **Histogram**
- Gauge
- Summary

Visualization in Grafana



Query 1 Transform 0 Alert 0

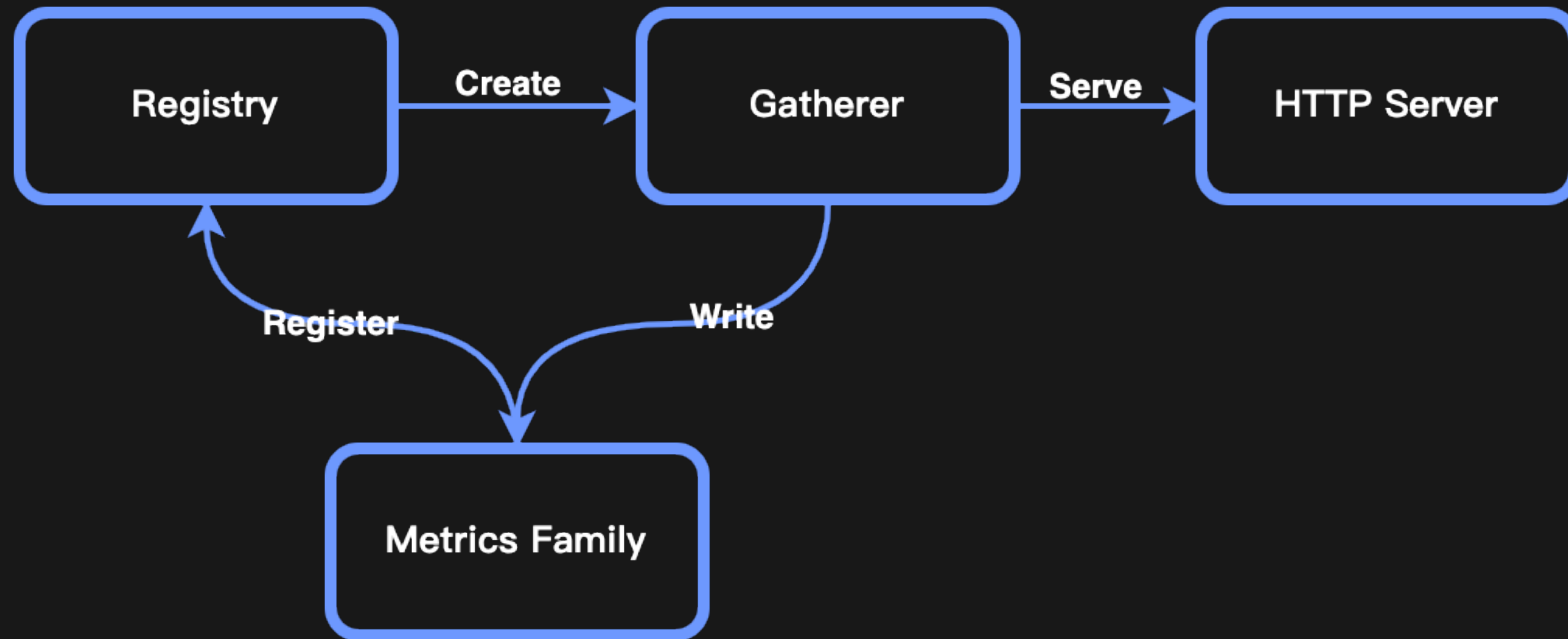
Prometheus-1-thanos ? > Query options MD = auto = 1342 Interval = 15s Query inspector

▼ A ? ? ? ? ?

Metrics ▼ histogram_quantile(0.90, sum(rate(gdb_dbm_transaction_seconds_bucket{service=~"\$service"}[1m])) by (le, tx_status)) * 1000

Legend ⓘ {{tx_status}} Min step ⓘ Resolution 1/1 ▼ Format Time series ▼ Instant Promethues ⓘ Exemplars

Prometheus Client



github.com/prometheus/client_golang

Implementation

```
var grpcServerMetrics = &GrpcServerMetrics{
    handledHistogramVec: promauto.NewHistogramVec(
        prometheus.HistogramOpts{
            Subsystem: subsystem,
            Name:      "grpc_server_handling_seconds",
            Help:      "Histogram of response latency (seconds) of gRPC that had been applica",
            Buckets:   defaultBuckets,
        }, []string{"app", "grpc_type", "grpc_service", "grpc_method", "grpc_status_code"}),
    streamMsgSentVec: promauto.NewHistogramVec(
```

```
func (m *GrpcServerMetrics) Handled(appName, svcType, svcName, method, code string, dur time.Duration) {
    ob := mustGetObserver(m.handledHistogramVec, appName, svcType, svcName, method, code)
    ob.Observe(dur.Seconds())
}
```

go.planetmeican.com/nerds/bazaar/xtelemetry

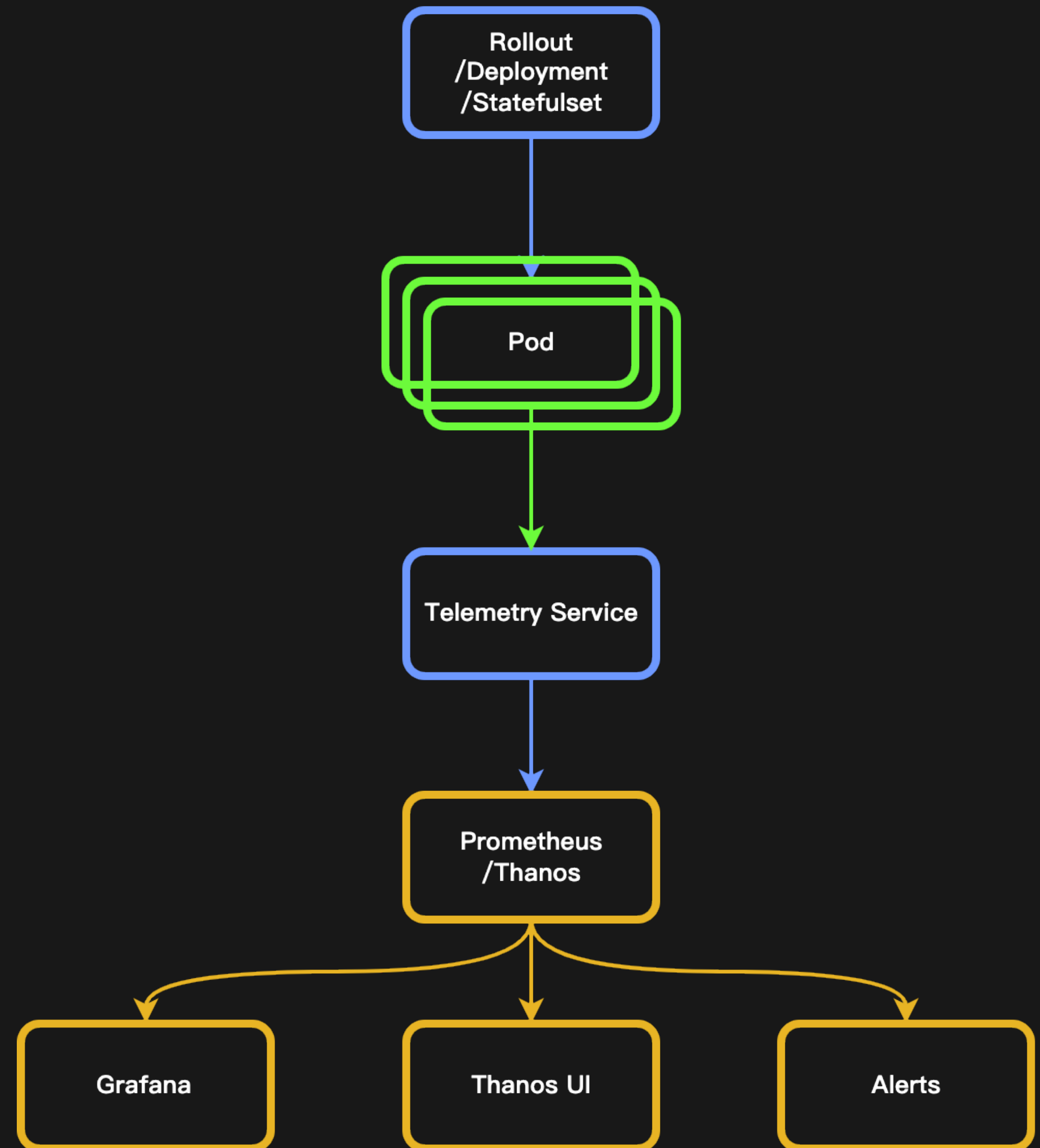
Implementation

```
// StartServer start a telemetry server in another go routine
func (tele *Telemetry) StartServer() error {
    addr := fmt.Sprintf(format: "%d", tele.port)
    listener, err := net.Listen(network: "tcp", addr)
    if err != nil {
        tele.Logger.Fatal(fmt.Sprintf(format: "net listen address %s error", addr),
            log.ErrField(err))
    }
    tele.Logger.Info(msg: "starting telemetry server", log.String(key: "addr", addr), log.Stri

    mux := http.NewServeMux()
    mux.Handle(metricsEndpoint, promhttp.Handler())
    if tele.nprofEnabled {
```

go.planetmeican.com/nerds/bazaar/xtelemetry

Metric Collection



Service Monitor

```

ports:
  - name: grpc
    containerPort: 8023
  - name: http-grpcgw
    containerPort: 8024
  - name: metrics
    containerPort: 18888

```

rollout.yaml

```

80 kind: Service
81 apiVersion: v1
82 metadata:
83   name: cmdb-server-telemetry
84   namespace: app-titan
85   labels:
86     app: cmdb
87     subset: server
88     stage: telemetry
89 spec:
90   selector:
91     app: cmdb
92     subset: server
93   ports:
94     - name: http-telemetry
95       protocol: TCP
96       port: 18888
97       targetPort: 18888

```

service.yaml

```

1  apiVersion: monitoring.coreos.com/v1
2  kind: ServiceMonitor
3  metadata:
4    labels:
5      name: cmdb
6      prometheus: k8s-thanos-2
7      name: cmdb-server-metrics-telemetry-monitor
8    namespace: monitoring
9  spec:
10   endpoints:
11     - port: http-telemetry
12       namespaceSelector:
13         matchNames:
14           - app-titan
15       selector:
16         matchLabels:
17           app: cmdb
18           subset: server
19           stage: telemetry

```

service-monitor.yaml

Usage

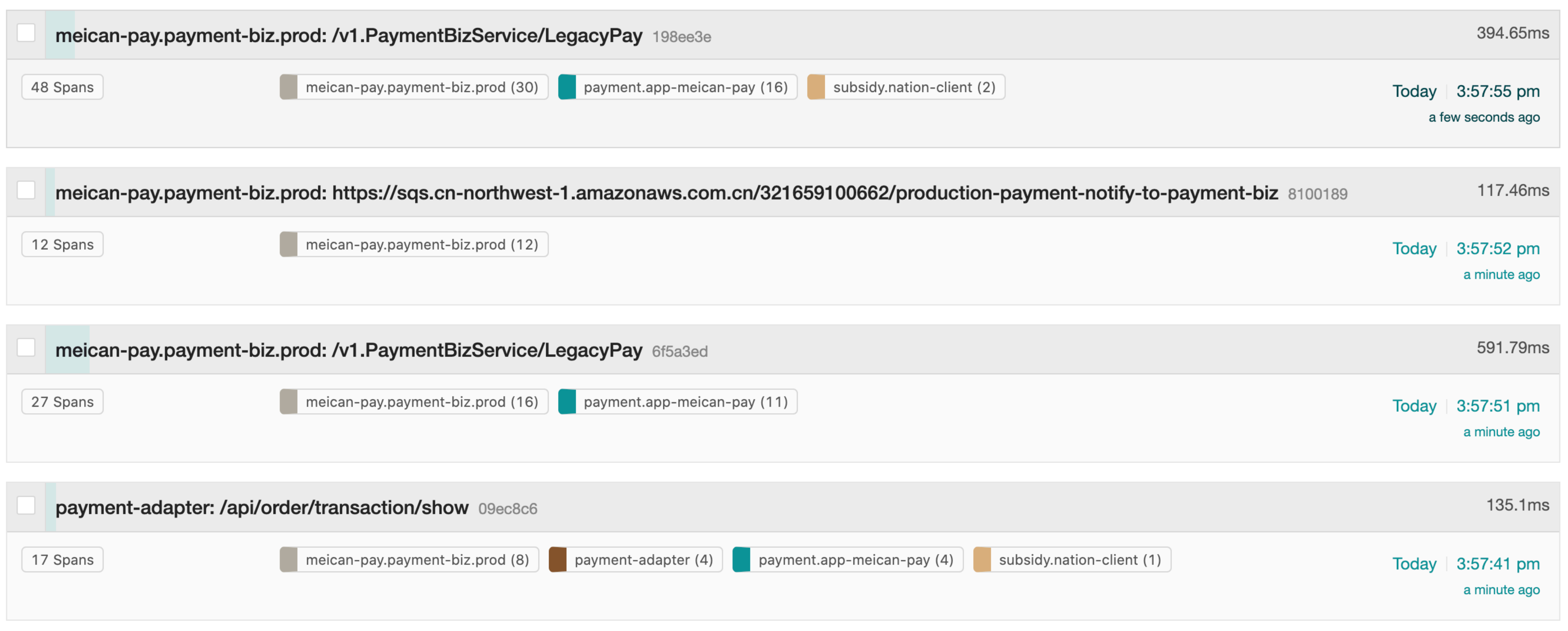
- Most go libraries already have metrics implemented:
 - go.planetmeican.com/nerds/...
- Bring your own:
 - go.planetmeican.com/nerds/bazaar/xtelemetry

Best practices

- Pay attention to labels:
 - {grpc_method} x {grpc_service} x {grpc_status_code} x 10 (buckets)
 - Results to exponential growth
 - Metrics count more than 5000(?) will be dropped by service monitor
- Histogram already contains sum and count
- It cannot represent detailed data. Don't expect 100% accuracy.

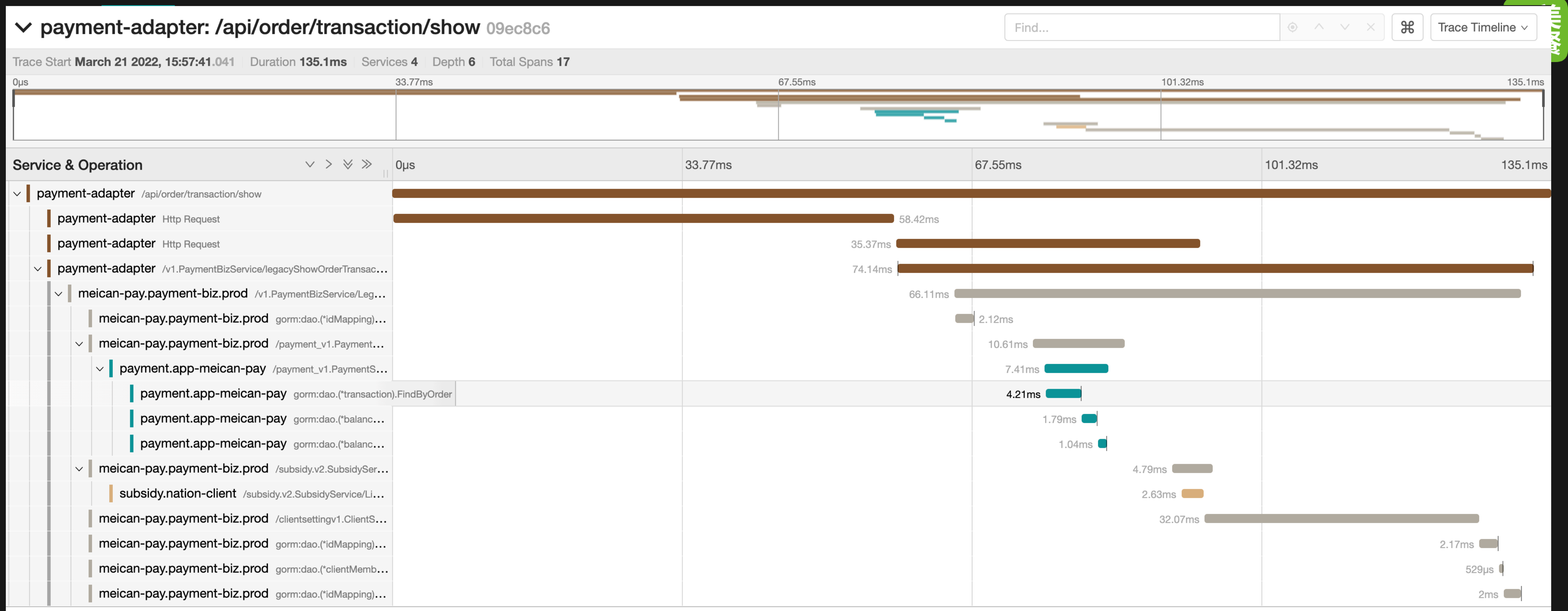
Traces

By Jaeger



Jaeger UI

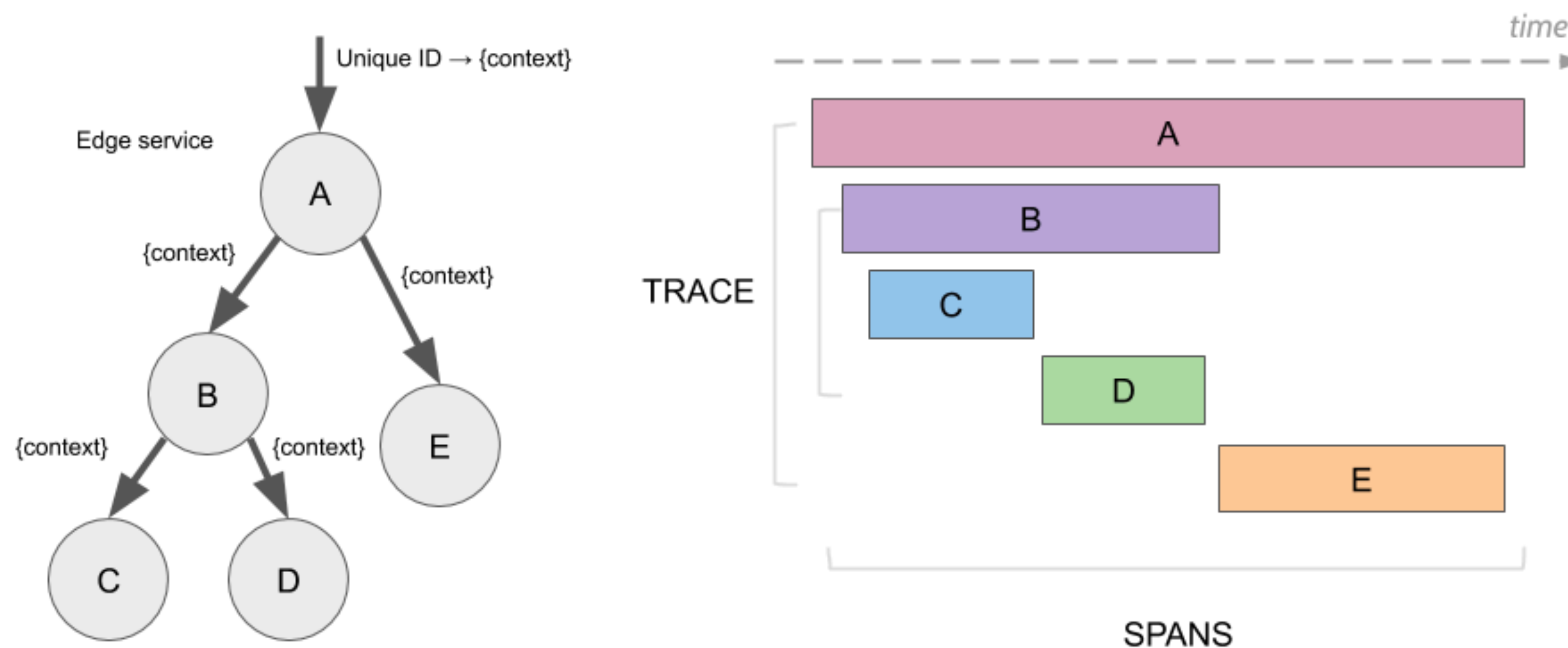
<https://grafana-nw.planetmeican.com>



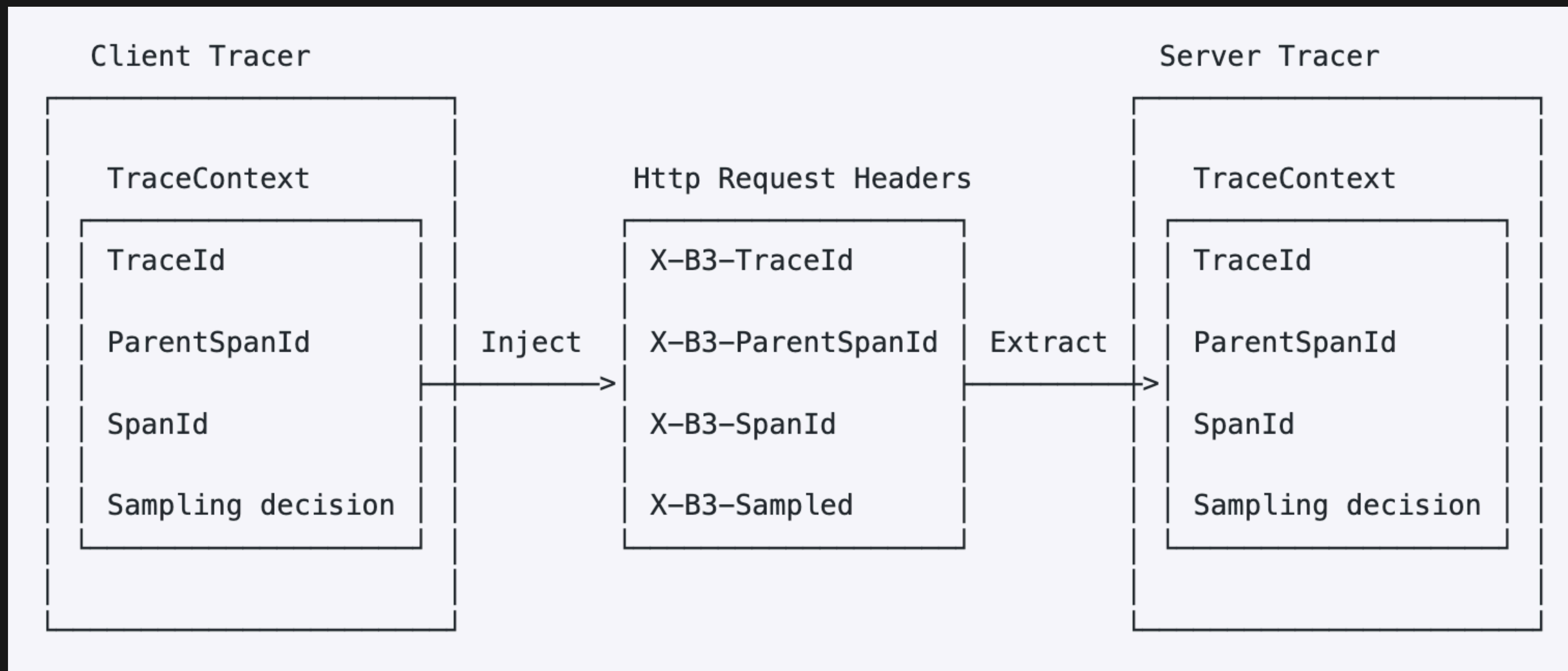
Jaeger UI

<https://grafana-nw.planetmeican.com>

Terminology



{trace-id}:{span-id}:{parent-span-id}:{flags}



Context Propagation

HTTP Headers (gRPC as well)

Client Side - HTTP

```
func makeSomeRequest(ctx context.Context) ... {
    if span := opentracing.SpanFromContext(ctx); span != nil {
        httpClient := &http.Client{}
        httpReq, _ := http.NewRequest("GET", "http://myservice/", nil)

        // Transmit the span's TraceContext as HTTP headers on our
        // outbound request.
        opentracing.GlobalTracer().Inject(
            span.Context(),
            opentracing.HTTPHeaders,
            opentracing.HTTPHeadersCarrier(httpReq.Header))

        resp, err := httpClient.Do(httpReq)
        ...
    }
    ...
}
```

Server Side - HTTP

```
http.HandleFunc("/", func(w http.ResponseWriter, req *http.Request) {
    var serverSpan opentracing.Span
    appSpecificOperationName := ...
    wireContext, err := opentracing.GlobalTracer().Extract(
        opentracing.HTTPHeaders,
        opentracing.HTTPHeadersCarrier(req.Header))
    if err != nil {
        // Optionally record something about err here
    }

    // Create the span referring to the RPC client if available.
    // If wireContext == nil, a root span will be created.
    serverSpan = opentracing.StartSpan(
        appSpecificOperationName,
        ext.RPCServerOption(wireContext))

    defer serverSpan.Finish()

    ctx := opentracing.ContextWithSpan(context.Background(), serverSpan)
    ...
})
```

Client Side - gRPC

```
// UnaryClientInterceptor returns a new unary client interceptor for OpenTracing.
func UnaryClientInterceptor(opts ...Option) grpc.UnaryClientInterceptor {
    o := evaluateOptions(opts)
    return func(parentCtx context.Context, method string, req, reply interface{}, cc *grpc.ClientConn,
        _ ...CallOption) error {
        if o.filterOutFunc != nil && !o.filterOutFunc(parentCtx, method) {
            return invoker(parentCtx, method, req, reply, cc, opts...)
        }
        newCtx, clientSpan := newClientSpanFromContext(parentCtx, o.tracer, method)
        if o.unaryRequestHandlerFunc != nil {
            o.unaryRequestHandlerFunc(clientSpan, req)
        }
        err := invoker(newCtx, method, req, reply, cc, opts...)
        finishClientSpan(clientSpan, err)
        return err
    }
}
```

```
func newClientSpanFromContext(ctx context.Context, tracer opentracing.Tracer, fullMethodName string) (context.Context,
    var parentSpanCtx opentracing.SpanContext
    if parent := opentracing.SpanFromContext(ctx); parent != nil {
        parentSpanCtx = parent.Context()
    }

    opts := []opentracing.StartSpanOption{
        opentracing.ChildOf(parentSpanCtx),
        ext.SpanKindRPCClient,
        grpcTag,
    }

    if tagx := ctx.Value(clientSpanTagKey{}); tagx != nil {
        if opt, ok := tagx.(opentracing.StartSpanOption); ok {
            opts = append(opts, opt)
        }
    }

    clientSpan := tracer.StartSpan(fullMethodName, opts...)
    // Make sure we add this to the metadata of the call, so it gets propagated:
    md := metaultils.ExtractOutgoing(ctx).Clone()
    if err := tracer.Inject(clientSpan.Context(), opentracing.HTTPHeaders, metadataTextMap(md)); err != nil {
        grpclog.Infof(format: "grpc_opentracing: failed serializing trace information: %v", err)
    }

    ctxWithMetadata := md.ToOutgoing(ctx)
    return opentracing.ContextWithSpan(ctxWithMetadata, clientSpan), clientSpan
}
```

Server Side - gRPC

```
// UnaryServerInterceptor returns a new unary server interceptor for OpenTracing.
func UnaryServerInterceptor(opts ...Option) grpc.UnaryServerInterceptor {
    o := evaluateOptions(opts)
    return func(ctx context.Context, req interface{}, info *grpc.UnaryServerInfo, handler grpc.UnaryHandler) (interface{}, error) {
        if o.filterOutFunc != nil && !o.filterOutFunc(ctx, info.FullMethod) {
            handler(ctx, req)
        }
        newCtx, serverSpan := newServerSpanFromInbound(ctx, o.tracer, o.traceHeaderName, info.FullMethod)
        if o.unaryRequestHandlerFunc != nil {
            o.unaryRequestHandlerFunc(serverSpan, req)
        }
        resp, err := handler(newCtx, req)
        finishServerSpan(ctx, serverSpan, err)
        return resp, err
    }
}
```

Server Side - gRPC

```
func newServerSpanFromInbound(ctx context.Context, tracer opentracing.Tracer, traceHeaderName, fullMethodName string) (context.Context, error) {
    md := metautils.ExtractIncoming(ctx)
    parentSpanContext, err := tracer.Extract(opentracing.HTTPHeaders, metadataTextMap(md))
    if err != nil && err != opentracing.ErrSpanContextNotFound {
        grpclog.Infof(format: "grpc_opentracing: failed parsing trace information: %v", err)
    }

    serverSpan := tracer.StartSpan(
        fullMethodName,
        // this is magical, it attaches the new span to the parent parentSpanContext, and creates an unparented one if empty.
        ext.RPCServerOption(parentSpanContext),
        grpcTag,
    )

    injectOpentracingIdsToTags(traceHeaderName, serverSpan, grpc_ctxtags.Extract(ctx))
    return opentracing.ContextWithSpan(ctx, serverSpan), serverSpan
}
```

Go Context

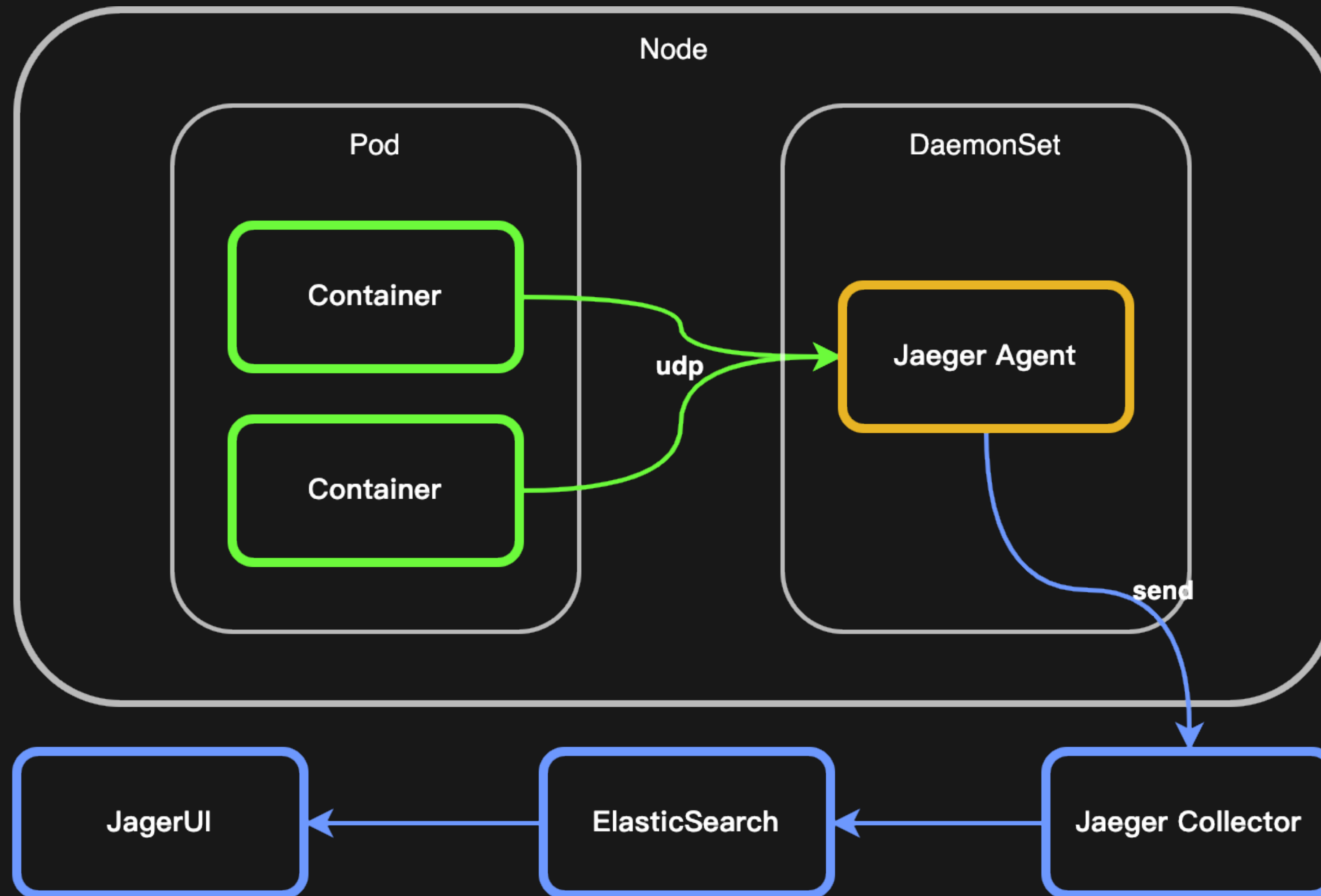
```
func SpanFromContext(ctx context.Context) Span {
    val := ctx.Value(activeSpanKey)
    if sp, ok := val.(Span); ok {
        return sp
    }
    return nil
}
```

Get

```
func ContextWithSpan(ctx context.Context, span Span) context.Context {
    if span != nil {
        if tracerWithHook, ok := span.Tracer().(TracerContextWithSpanExtension); ok {
            ctx = tracerWithHook.ContextWithSpanHook(ctx, span)
        }
    }
    return ctx.WithValue(activeSpanKey, span)
}
```

Set

Trace Collection



Best Practices

- Chain upstream - downstream
 - Interceptor (middleware) on both client and server sides
 - Extract incoming -> inject outgoing
- Standardized tag and naming conventions:
 - go.opentelemetry.io/contrib/instrumentation/...
 - github.com/opentracing/opentracing-go/ext

Standardized Tag

```
if err != nil {
    ext.Error.Set(serverSpan, value: true)
    serverSpan.LogFields(log.String(key: "event", val: "error"), log.String(key: "message", err.Error()))
}
```

Example

```
// HTTPUrl should be the URL of the request being handled in this segment
// of the trace, in standard URI format. The protocol is optional.
HTTPUrl = StringTagName("http.url")

// HTTPMethod is the HTTP method of the request, and is case-insensitive.
HTTPMethod = StringTagName("http.method")

// HTTPStatusCode is the numeric HTTP status code (200, 404, etc) of the
// HTTP response.
HTTPStatusCode = Uint16TagName("http.status_code")
```

```
span = tracer.StartSpan(operationName, ext.RPCServerOption(wireContext))
ext.HTTPMethod.Set(span, req.Method)
ext.HTTPUrl.Set(span, req.URL.String())
```

Example

Best Practices

- Setting “sampling.priority”:

- Prod 1%

```
span := opentracing.SpanFromContext(ctx)
ext.SamplingPriority.Set(span, value: 999)
```

- Better not set: ~~JAEGER_SAMPLER_TYPE~~ and ~~JAEGER_SAMPLER_PARAM~~
- Use remote sampling strategy

Remote Sampling Configurable

```
sampling:
  options:
    service_strategies:
      - service: jaeger-example
        type: probabilistic
        param: 0.1
      operation_strategies:
        - operation: /getTime
          type: probabilistic
          param: 0.2
        - operation: /drivers
          type: probabilistic
          param: 0.4
    default_strategy:
      type: probabilistic
      param: 0.2
    operation_strategies:
      - operation: /health-check
        type: probabilistic
        param: 0.0
      - operation: /metrics
        type: probabilistic
        param: 0.0
      - operation: /healthz
        type: probabilistic
        param: 0.0
```

Special Notice

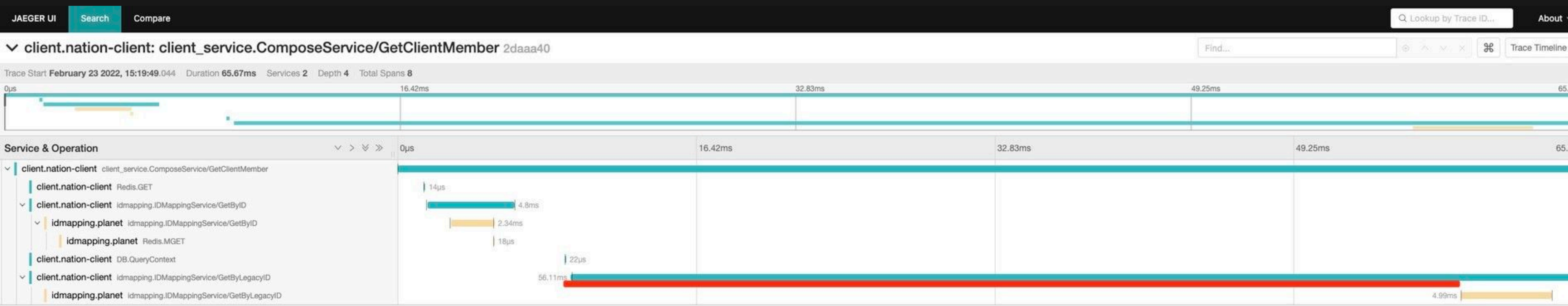
- OpenTracing is deprecated, use OpenTelemetry instead
 - [OOPS-644](#)

Case #1 : Network Issue ?

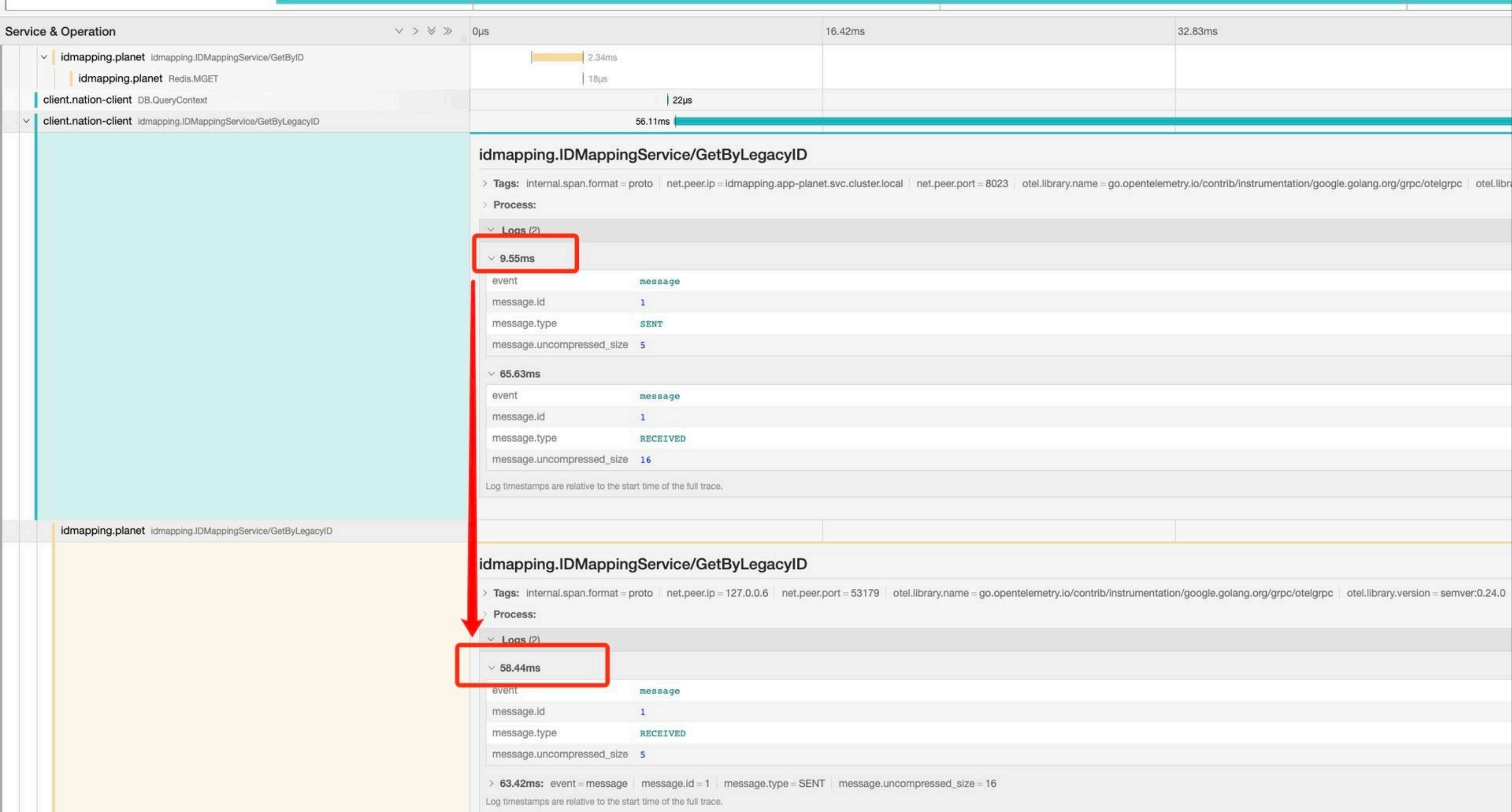


- Benchmarking
- gRPC
- Suspicious trace

Case #1: Network Issue ?



Case #1: Network Issue ?

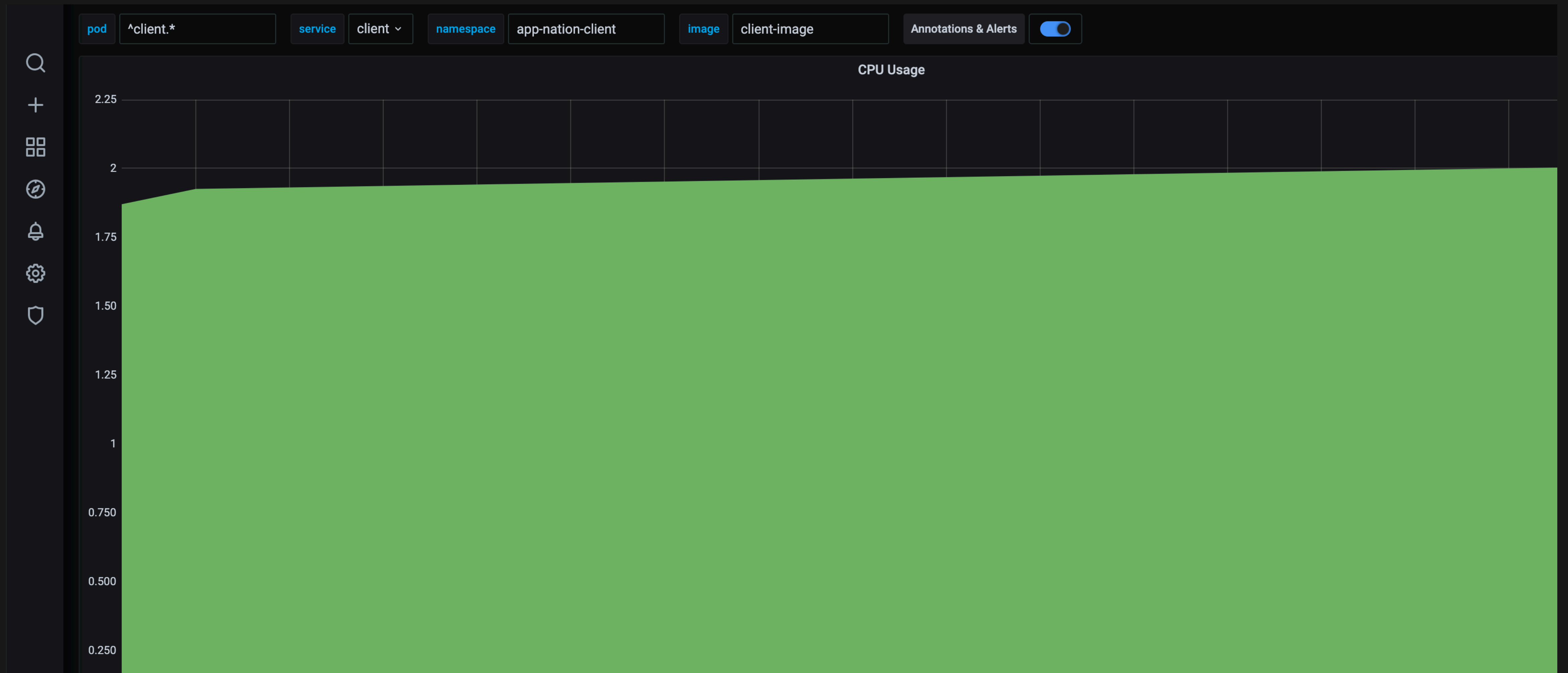


Case #1: DNS is okay

Responses (latency, internet zone)



Case #1: High CPU Utilization



Case #1: Resolved

By setting rate limit

Question?

The End.

Next: Closed Loop Alerting (CLA)